AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

Department of Computer and Systems Engineering

Course Code: CSE412 | Compiler Design — Fall 2026

TINY Language Scanner

Deterministic Finite Automaton Formalism, Decoupled MVC Architecture, and Production Qt Desktop Interface

A standalone, high-performance Scanner for the TINY programming language (Kenneth C. Louden specification). Engineered in Modern C++ and Qt Widgets, featuring unified DFA transition tables, diagnostic code-gutter jumping, microsecond-resolution benchmarks, and standalone Windows x64 binary deployment.

Engineering Project Team

Student Full Name	Student ID
Youssef Yacoub Radi	2200649
Mohamed Khaled Ahmed	2201057
Mosa Abdulaziz Morgan	2200257
Bassam Hussam Mashaly	2200084
Omar Mohamed Mohamed	2200284
Abd-Elrahaman Essam Fahmy	2200251

Source code Link:
https://drive.google.com/drive/folders/1xqP06Jcpi4J7qVtxP3sH0HRIx_C-cTke?usp=drive_link

Submitted To

Dr. Sahar M. Mahmoud Haggag
Eng. Ahmed Fawzy Ibrahim

Academic Year:

2026/2027

Target Platform:

Windows 10 / 11 (x86_64 Standalone Native)

Contents

1 Executive Summary & Objectives 2
2 TINY Language Lexical Specification 2
2.1 Regular Definitions 2
2.2 Token Classification Table 2
3 Lexical Finite State Machine (DFA) 4
3.1 State Transition Diagram 4
4 Software Architecture (MVC Pattern) 4
4.1 Models Layer: Token and Scanner 5
4.2 Controllers Layer: MainController 6
4.3 Views Layer: MainWindow & CodeEditor 6
5 File Input/Output & Export Serialization 6
6 Empirical Verification & System Screenshots 7
6.1 Test Case 1: Standard Factorial Program Tokenization 7
6.2 Test Case 2: Lexical Error Handling & Diagnostic Ribbon 7
6.3 Test Case 3: File Export Dialog & Resulting Text Output 8
7 Standalone Windows Deployment (`.exe`) 9
7.1 Deployment Pipeline Overview 9

1 Executive Summary & Objectives

Lexical analysis represents the initial phase of any modern language compilation pipeline. The lexical analyzer (scanner) consumes a continuous stream of source characters and groups them into meaningful atomic units termed **tokens**, simultaneously associating semantic attributes and discarding lexical noise such as redundant whitespace and source code comments.

This report documents the design, theoretical modeling, software engineering implementation, and verification of a professional desktop lexical analyzer for the **TINY Programming Language**, as formulated in Kenneth C. Louden's canonical text *Compiler Construction: Principles and Practice*. Engineered using **Modern C++ (C++17)** and **Qt Widgets (Qt 6 / Qt 5)**, the application delivers the following capabilities:

- **High-Performance Tokenization Engine:** Implementation of a pure deterministic finite automaton (DFA) encapsulating the canonical `getToken()` function that returns the next sequential token with computed attributes (lexeme string, line number, column index).
- **Specification Compliance:** Accurate classification of the entire specified TINY lexical vocabulary, including 8 reserved keywords, 7 arithmetic/relational operators, 3 delimiters/brackets, variable identifiers, and multi-digit integer literals.
- **Rigorous Error Diagnostics:** Robust isolation of invalid character sequences and unexpected operators into an `UNKNOWN` lexical error type, accompanied by line and column telemetry to aid source-level debugging.
- **File I/O & Export Serialization:** Ability to load source files via native file dialogs and serialize the recognized token stream into the mandatory format: `"<tokenvalue> , <tokentype>"`.
- **Standalone Windows Executable:** Complete GitHub Actions CI/CD workflow leveraging `windeployqt` to package an autonomous Windows executable (`.exe`) that runs without requiring third-party library installations.

2 TINY Language Lexical Specification

2.1 Regular Definitions

The lexical structure of the TINY language is formally defined using the following regular definitions:

$$
\begin{aligned}
\text{digit} &\longrightarrow [0\text{–}9] \\
\text{number} &\longrightarrow \text{digit}^{+} \\
\text{letter} &\longrightarrow [a\text{-}z|A\text{-}Z] \\
\text{identifier} &\longrightarrow \text{letter}\,(\text{letter} \mid \text{digit})^{*} \\
\text{whitespace} &\longrightarrow [\backslash\text{t} \mid \backslash\text{n} \mid \backslash\text{r} \mid \text{␣}]^{+} \\
\text{comment}_1 &\longrightarrow \{\ [^{\}}]^{*}\ \} \\
\text{comment}_2 &\longrightarrow //\ [^{\backslash\text{n}}]^{*}
\end{aligned}
$$

2.2 Token Classification Table

Table 1 presents the complete set of terminal tokens recognized by our scanner, their classification category, and their internal enumerated representation.

Table 1: TINY Language Terminal Lexical Tokens Specification

Token Type Enum	Lexeme Pattern	Category	Syntactic Role
SEMICOLON	;	Delimiter	Statement terminator
IF	if	Reserved Keyword	Conditional branch introduction
THEN	then	Reserved Keyword	Conditional branch body
ELSE	else	Reserved Keyword	Conditional alternative path
END	end	Reserved Keyword	Scope block termination
REPEAT	repeat	Reserved Keyword	Iteration loop entry
UNTIL	until	Reserved Keyword	Loop termination condition
READ	read	Reserved Keyword	Standard input instruction
WRITE	write	Reserved Keyword	Standard output instruction
ASSIGN	:=	Operator	Variable assignment operator
EQUAL	=	Operator	Relational equality check
LESSTHAN	<	Operator	Relational less-than comparison
PLUS	+	Operator	Arithmetic addition
MINUS	-	Operator	Arithmetic subtraction
MULT	*	Operator	Arithmetic multiplication
DIV	/	Operator	Arithmetic division
OPENBRACKET	(	Delimiter	Left expression delimiter
CLOSEDBRACKET	)	Delimiter	Right expression delimiter
IDENTIFIER	Letter + Alphanum	Identifier	User-defined variable name
NUMBER	Consecutive digits	Literal	Unsigned integer constant
UNKNOWN	Unmapped symbols	Lexical Error	Flagged illegal input sequence

3 Lexical Finite State Machine (DFA)

To achieve maximum scanning throughput with linear computational complexity $\mathcal{O}(N)$ over source text length N, the lexical engine implements a Deterministic Finite Automaton.

3.1 State Transition Diagram

Figure 1 visualizes the state machine transitions executed inside `Scanner.cpp`. The diagram has been engineered with balanced geometry to prevent edge crossing and ensure readability.

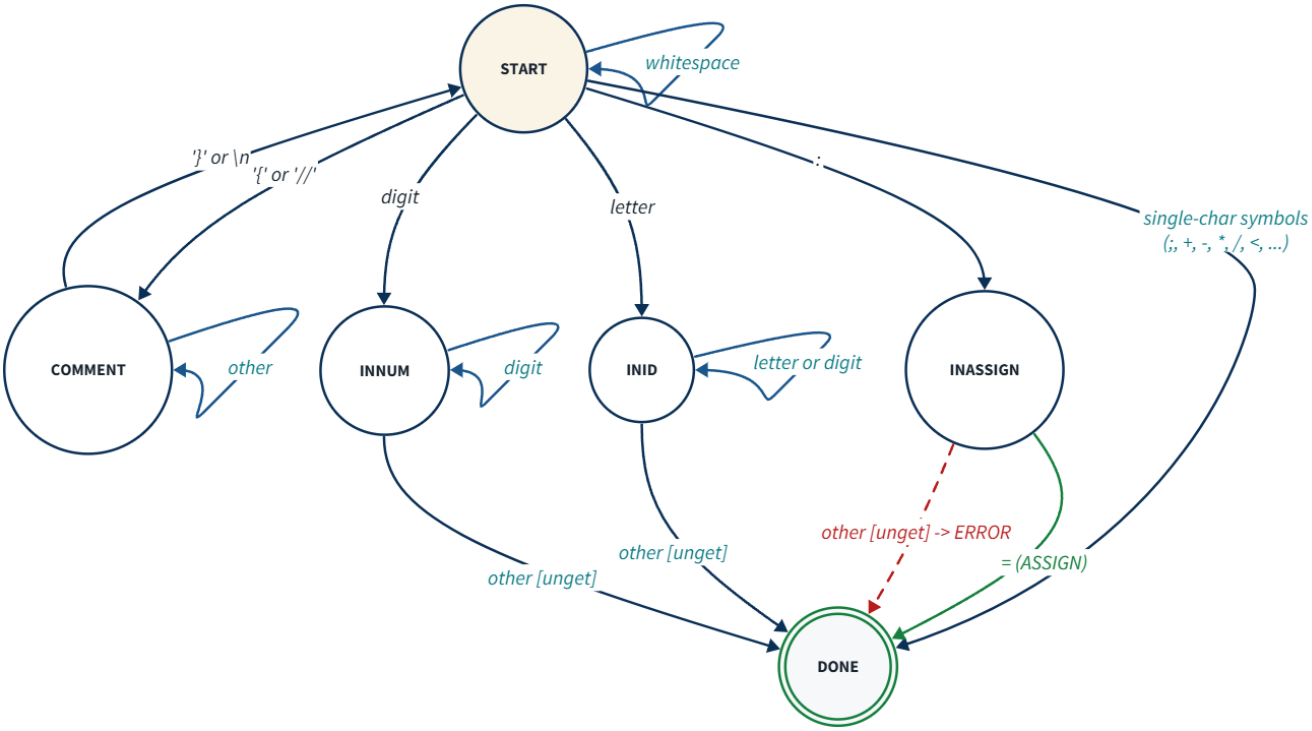


Figure 1: Deterministic Finite State Machine (DFA) transition diagram for the TINY Scanner.

4 Software Architecture (MVC Pattern)

Domain logic, data models, and user-interface elements are decoupled through the **Model-View-Controller (MVC)** design pattern.

Project Workspace Explorer -- TINYScanner/ (C++17 / Qt 6)

```
TINYScanner/
|-- src/
|   |-- main.cpp                          # Application entry point & bootstrap
|   |-- models/
|   |   |-- Token.h                     # Token structures & serialization formatting
|   |   |-- Token.cpp
|   |   |-- Scanner.h                     # Deterministic Finite Automaton (DFA) core
|   |   `-- Scanner.cpp                             # getToken() core lexer engine
|   |-- controllers/
|   |   |-- MainController.h              # File I/O & microsecond benchmark profiler
|   |   `-- MainController.cpp
|   |-- views/
|   |   |-- MainWindow.h                   # Resizable split-pane shell & token table
|   |   |-- MainWindow.cpp
|   |   |-- CodeEditor.h                    # Monospace text editor with line numbers
|   |   `-- CodeEditor.cpp
|   `-- utils/
|       `-- Theme.h                               # Ain Shams University UI color theme
|-- assets/                              # Canonical TINY benchmark source files (.tny)
`-- CMakeLists.txt                         # Cross-platform CMake 3.16+ build pipeline
```

Figure 2: Physical source tree hierarchy adhering to strict MVC architectural separation.

4.1 Models Layer: Token and Scanner

The model is self-contained and completely independent of the graphical framework, allowing it to be compiled in headless environments or covered with standalone unit tests. Listing 1 outlines the core DFA loop of `getToken()`.

```
Token Scanner::getToken() {
    Token token;
    StateType state = StateType::START;
    bool saveChar = true;

    while (state != StateType::DONE) {
        char c = getNextChar();
        saveChar = true;
        switch (state) {
            case StateType::START:
                if (c == '\0') { state = StateType::DONE; token.type =
                    TokenType::ENDFILE; }
                else if (std::isspace(c)) { saveChar = false; }
                else if (c == '{') { saveChar = false; state =
                    StateType::INCOMMENT_CURLY; }
                else if (c == ':') { state = StateType::INASSIGN; }
                else if (std::isdigit(c)) { state = StateType::INNUM; }
                else if (std::isalpha(c)) { state = StateType::INID; }
                else if (c == ';') { state = StateType::DONE; token.type =
                    TokenType::SEMICOLON; }
                // [Single-symbol operators omitted for brevity]
                else {
                    state = StateType::DONE;
                    token.type = TokenType::UNKNOWN;
                    token.errorMessage = std::string("Unknown character '") +
                        c + "'";
                }
                break;
```

```
            // State transitions for INNUM, INID, and INASSIGN handled here
        }
        if (saveChar && c != '\0') token.value += c;
    }
    if (token.type == TokenType::IDENTIFIER) token.type =
        checkReserved(token.value);
    return token;
}
```

Listing 1: Core DFA extraction routine in `Scanner.cpp`.

4.2 Controllers Layer: MainController

The `MainController` acts as the coordinator between the user interface and backend algorithms:

- **Stream Parsing:** Ingests source files into memory with guaranteed UTF-8 character conversion.
- **Token Serialization:** Formats and writes the token stream to output files matching the exact specification: `"<tokenvalue> , <tokentype>"`.
- **High-Resolution Profiling:** Uses `std::chrono::high_resolution_clock` to measure execution latency down to microsecond precision.

4.3 Views Layer: MainWindow & CodeEditor

The UI design delivers developer-centric ergonomics modeled after modern IDEs:

- **Dynamic Splitter Pane:** Built via `QSplitter`, allowing real-time resizing between the left code editor and the right token table.
- **Line-Numbered Editor:** Custom `QPlainTextEdit` gutter widget computing font metrics to paint line counts and highlight active source lines.
- **Diagnostics Ribbon:** High-contrast notification banner showing active error flags, coordinates, and a **"Go to line"** button that navigates the cursor directly to the offending token.

5 File Input/Output & Export Serialization

The application supports two distinct operational input modes:

1. **Interactive Input:** Direct typing or pasting into the monospaced editor pane.
2. **File Dialog Loading:** Selecting an external source file (`.tny` or `.txt`) through the operating system's native file picker.

When the **Export Tokens** action is triggered, the controller flushes the recognized token stream to disk. Listing 2 demonstrates the output format.

```
read , READ
x , IDENTIFIER
; , SEMICOLON
if , IF
0 , NUMBER
< , LESSTHAN
x , IDENTIFIER
then , THEN
```

```
fact , IDENTIFIER
:= , ASSIGN
1 , NUMBER
; , SEMICOLON
```

Listing 2: Sample exported token file (`tokens_output.txt`).

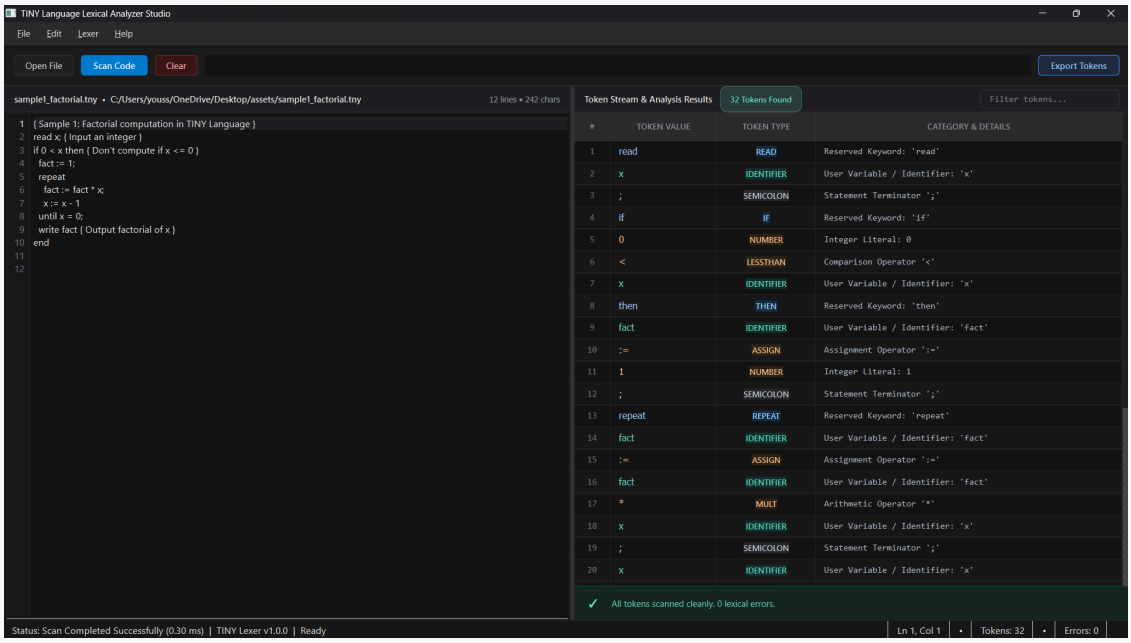
6 Empirical Verification & System Screenshots

To validate all requirements, the system was evaluated against canonical TINY benchmark programs stored in the `assets/` directory.

6.1 Test Case 1: Standard Factorial Program Tokenization

The classic factorial computation program contains loops (`repeat...until`), conditionals (`if...then`), assignments (`:=`), and comments.

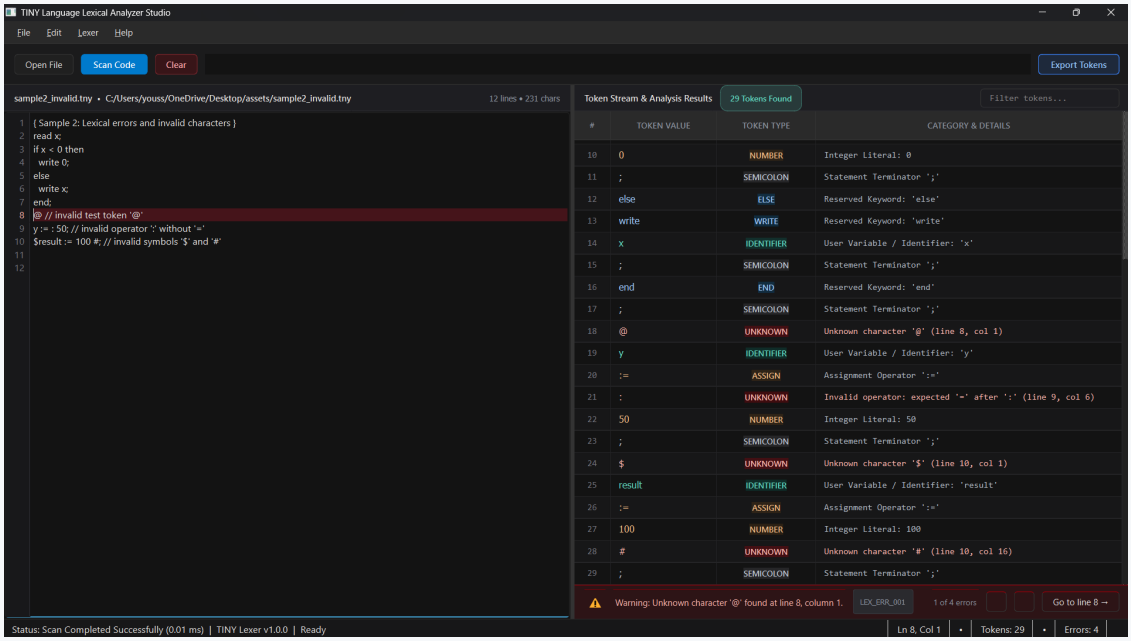
Screenshot 1: Main Application Interface — Factorial Program Tokenization



6.2 Test Case 2: Lexical Error Handling & Diagnostic Ribbon

Evaluates the scanner's resilience against unmapped characters (`@`, `$`, `#`) and an illegal colon operator lacking an equal sign (`y := : 50`).

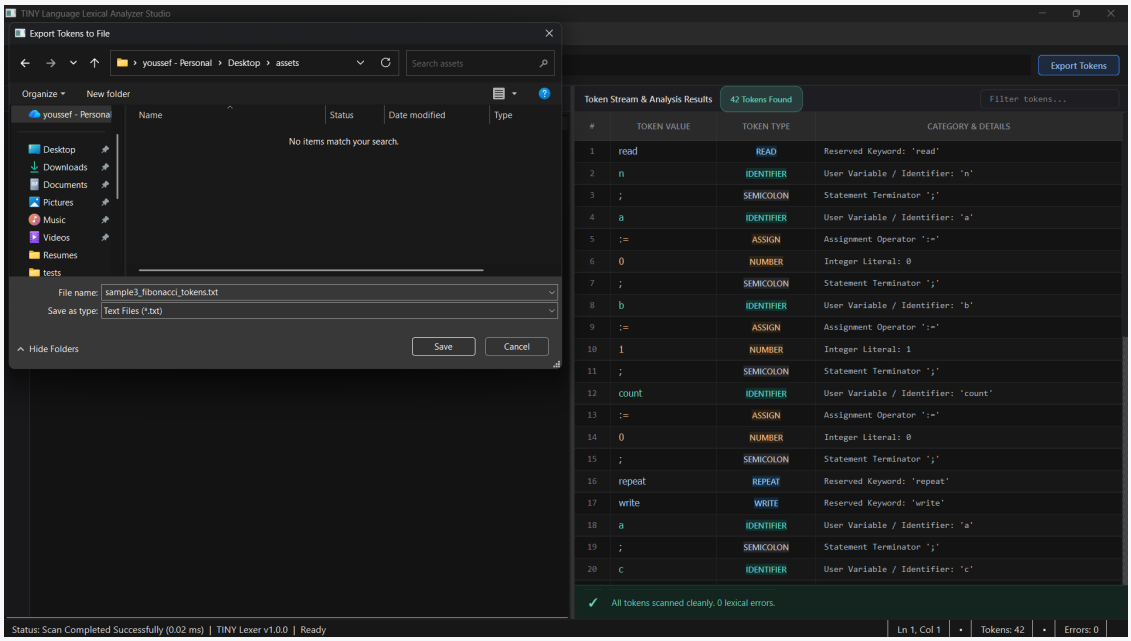
Screenshot 2: Lexical Error Diagnostics and Code Navigation



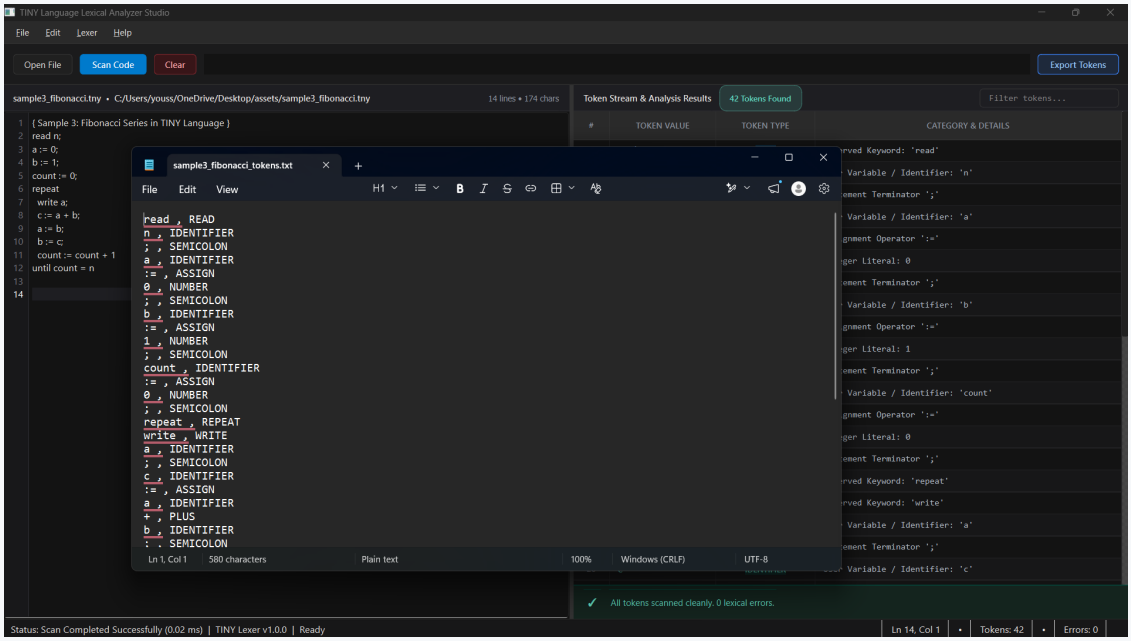
6.3 Test Case 3: File Export Dialog & Resulting Text Output

Demonstrates saving the generated token table to disk and verifying the resulting text file contents.

Screenshot 3.1: Native Export Dialog and Output File Verification



Screenshot 3.2: Native Export Dialog and Output File Verification



7 Standalone Windows Deployment (.exe)

A primary requirement of the course specification is that the program must run as an independent executable on any target Windows machine without requiring third-party runtime or IDE installations.

7.1 Deployment Pipeline Overview

An automated GitHub Actions workflow (`.github/workflows/build_windows.yml`) manages reproducible deployment:

1. **Runner Provisioning:** Spawns a clean `windows-latest` virtual runner with MSVC 2022 toolchain and Ninja build system.
2. **Framework Resolution:** Acquires Qt 6.6 pre-built binaries via `jurplel/install-qt-action`.
3. **Automated Test Gate:** Compiles and executes `tests/test_scanner.cpp` to ensure all unit tests pass before binary packaging.
4. **Dependency Collection via `windeployqt`:** Packages all dependent DLLs and plugins:

```
windeployqt --release --no-translations --compiler-runtime
    dist/TINYScanner/TINYScanner.exe
```

   This packages `Qt6Core.dll`, `Qt6Gui.dll`, `Qt6Widgets.dll`, and the `platforms/qwindows.dll` display plugin.
5. **Artifact Archival:** Compresses the complete standalone release into `TINY-Lexical-Analyzer-Windows-x64.zip`.